

Un check `clang-tidy` da zero

Costruzione, test e integrazione di un linter C++ che capisce il codice, non solo il testo

Martin Trajkovski

Linguaggi, Interpreti e Compilatori · A.A. 2025/26

Il problema

Tutto nasce da un fastidio concreto in code review.

"Continuo a vedere `printf` in C++ moderno. Vorrei che il CI me lo segnali da solo, con il fix già pronto."

- I pattern che si ripetono andrebbero **controllati dalla macchina**, non dall'occhio umano
- Ma gli strumenti testuali (`grep` , espressioni regolari) leggono **caratteri**, non significato: si perdono o sbagliano appena entrano in gioco **scope, template, macro**
- Serve uno strumento che ragioni su **tipi e contesto**, come fa il compilatore

Lo strumento esiste già: si chiama clang-tidy. Vediamo com'è fatto dentro.

Architettura di clang-tidy

Il compilatore non è una scatola chiusa: è una libreria su cui possiamo agganciarci.



in **rosso**: gli unici due pezzi che scrivo io · tutto il resto è infrastruttura Clang

- Il **frontend** di Clang trasforma il sorgente in un **AST** (l'albero della struttura del programma) già arricchito di **semantica**: tipi risolti, overload scelti, template istanziati
- Io scrivo solo i **matcher** (quali nodi cercare) e il **check** (cosa fare quando li trovi)

Cos'è l'AST, in 4 righe

AST = Abstract Syntax Tree, l'albero che rappresenta la struttura del programma.

```
echo 'int x = (int)3.14;' | clang -x c++ -Xclang -ast-dump -fsyntax-only -
```

```
TranslationUnitDecl
├── VarDecl 'x' 'int' ← dichiarazione di variabile
│   ├── CStyleCastExpr 'int' <FloatingToIntegral> ← il cast, con tipo
│   │   └── FloatingLiteral 'double' 3.14 ← il valore 3.14
```

Il cast non è la stringa `"(int)"`: è un **nodo tipato** che sa di trasformare un `double` in un `int`. È su questo livello, non sul testo, che ragionano i check.

Gli AST Matchers: un piccolo linguaggio per cercare nodi

DSL = Domain-Specific Language, un mini-linguaggio dedicato a un solo compito.

```
// "ogni chiamata a una funzione il cui nome è ::printf"
callExpr(callee(functionDecl(hasName("::printf"))))

// "ogni metodo virtuale che non è un override"
cxxMethodDecl(isVirtual(), unless(isOverride()))
```

Si legge come una frase, dall'esterno verso l'interno, e si compone come un albero. È **dichiarativo**: descrivi *cosa* cercare, non *come* visitare l'albero. Molto più conciso che scrivere a mano un visitatore ricorsivo.

Anatomia di un check

Ogni check è una classe C++ con due soli "agganci".

```
class NoPrintfCheck : public ClangTidyCheck {  
public:  
    NoPrintfCheck(StringRef N, ClangTidyContext *C)  
        : ClangTidyCheck(N, C) {}  
    void registerMatchers(MatchFinder *F) override; // QUALI nodi cerco  
    void check(const MatchFinder::MatchResult &R) override; // COSA faccio  
};
```

È piccola di proposito: tutta la complessità (parsing, type checking, name resolution) la gestisce il driver di Clang. Io mi concentro solo sulla regola.

Il nostro check: `misc-no-printf`

Una regola minimale ma non banale, scelta apposta per toccare i casi difficili.

Cosa fa:

- segnala le chiamate a `::printf` di libc (la `printf` "vera", del C)
- **non** segnala `mylib::printf`: è un'altra funzione, sta in un altro **scope**
- emette un **fix-it**: sostituisce il nome con `std::print` (il rimpiazzo type-safe di C++23)

Perché è un buon esempio didattico:

- il matcher è di poche righe, ma deve gestire **scope**, **macro**, **template**
- il fix-it è chirurgico: tocca solo il nome, non l'intera chiamata

registerMatchers: dichiaro cosa cercare

Il cuore del check sta tutto in questo singolo matcher.

```
void NoPrintfCheck::registerMatchers(MatchFinder *F) {  
    F->addMatcher(  
        callExpr(  
            callee(functionDecl(hasName("::printf"))),  
            unless(isExpansionInSystemHeader()) // ignora gli header di sistema  
        ).bind("printfCall"),  
        this);  
}
```

- `hasName("::printf")` aggancia il nome **globale e qualificato** → nessuna collisione con `printf` definite dall'utente
- `unless(isExpansionInSystemHeader())` evita di segnalare codice che non è nostro (es. dentro `<stdio>`)

check: produco diagnostica e fix-it

Quando un nodo matcha, questa funzione viene chiamata e genera l'avviso.

```
void NoPrintfCheck::check(const MatchFinder::MatchResult &R) {  
    const auto *Call = R.Nodes.getNodeAs<CallExpr>("printfCall");  
    const Expr *Callee = Call->getCallee()->IgnoreImpCasts();  
    diag(Callee->getBeginLoc(),  
        "uso di 'printf' sconsigliato; usare 'std::print'")  
    << FixItHint::CreateReplacement(  
        SourceRange(Callee->getBeginLoc(), Callee->getEndLoc()),  
        "std::print");  
}
```

`FixItHint::CreateReplacement` sostituisce **solo** il token `printf` con `std::print`: il fix è applicabile in automatico, su mille file, a ogni PR.

Demo

Dagli esempi piccoli al progetto reale, e il confronto con gli LLM

Demo 1: esempi mirati + confronto con gli LLM

Prima dimostro che il check "capisce" davvero scope e overload, poi lo metto contro due LLM.

```
clang-tidy -checks='-* , misc-no-printf' snippets/02_namespace_collision.cpp
# → 1 hit: SOLO ::printf, ignora mylib::printf

clang-tidy -checks='-* , misc-no-printf' snippets/06_using_template_alias.cpp
# → 1 hit: l'overload resolution diretta ::printf("...", int) al template
```

Confronto: 6 snippet × {Claude Opus 4.7, ChatGPT 5.5} × 2 chat fresche = 24 run

	clang-tidy	Claude	ChatGPT
Determinismo	100%	alto su 5/6	medio
Fix primario = <code>std::print</code>	sì	4/6	0/6
Snippet 06 (overload res.)	sempre 1 hit	1 ^a ✓ / 2 ^a ✗	1 ^a ✗ / 2 ^a ✓
Bug fuori scope (<code>2.5→2</code>)	non visto	trovato ★	trovato ★

Il risultato più netto: stesso modello, due chat identiche → interpretazioni **opposte** dello stesso codice.

Demo 2: su progetti veri

Gli esempi piccoli convincono poco: ecco lo stesso check su codebase reali.

Caso A · `bear_demo` (build con Make):

```
bear -- make          # "registra" la build e produce compile_commands.json
clang-tidy -p . main.cpp logger.cpp net.cpp util.cpp
# → 5/5 hit, 0 falsi positivi
```

Caso B · `tinycl2` (libreria reale da GitHub, build CMake):

```
cmake -B build -DCMAKE_EXPORT_COMPILE_COMMANDS=ON
clang-tidy -p build --extra-arg=-isysroot $(xcrun --show-sdk-path) \
  tinycl2.cpp xmltest.cpp
# → 28 hit in 0.2s
```

Il numero da ricordare: `grep printf` → **42** match · `clang-tidy` → **28**.

I 14 di scarto sono commenti, stringhe, `vfprintf` / `snprintf`, macro. Quello è il valore dell'AST.

Test, documentazione, integrazione

Un check serio non è solo codice: è anche test riproducibili e integrazione in CI.

- **Test in stile lit** (il framework di test di LLVM): `RUN: %check_clang_tidy %s misc-no-printf %t`, gira dentro la build farm ufficiale, verifica messaggi e fix applicato
- **Documentazione** in `.rst`, come tutti i check ufficiali di LLVM
- **Integrazione CI** (Continuous Integration): `clang-tidy --warnings-as-errors='*'` blocca il merge se il check trova qualcosa
- **Distribuzione**: si compila dentro `clang-tools-extra` → può essere proposto upstream

Take-away

La domanda finale non è "chi vince", ma "che tipo di domanda si fa al codice".

Quando scrivere un check custom: pattern interno ricorrente · vincolo deterministico · fix riapplicabile in CI

Quando NO: regola sfumata (meglio review umana o LLM) · stile generico (esiste già un check ufficiale)

clang-tidy e LLM sono complementari, non sostituti:

- **clang-tidy** → deterministico e preciso sul suo mandato (scope, tipi, overload)
- **LLM** → trovano bug fuori dal mandato (`static_cast<int>(2.5) → 2`), ma sulla precisione dell'AST **fluttuano tra un'esecuzione e l'altra**

Se la domanda ha una risposta deterministica → vince clang-tidy.

Se richiede comprensione del contesto → vince l'LLM.

Risorse: [clang-tidy/Contributing.html](#) · [LibASTMatchersReference.html](#) · repo: [Martin/1_clang-tidy_custom_check/](#)